

High-Level Architecture

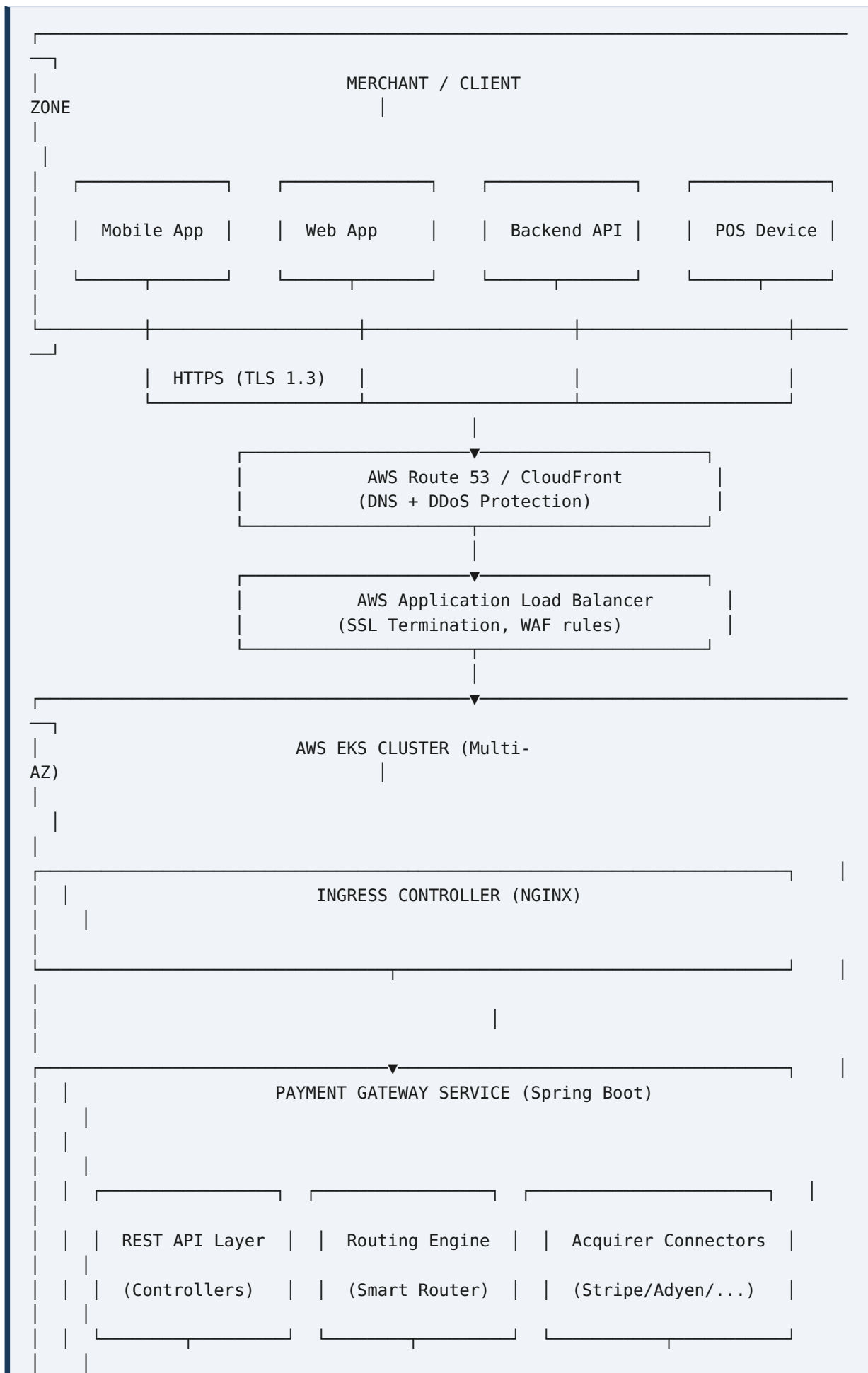
Multi-Acquirer Payment Gateway

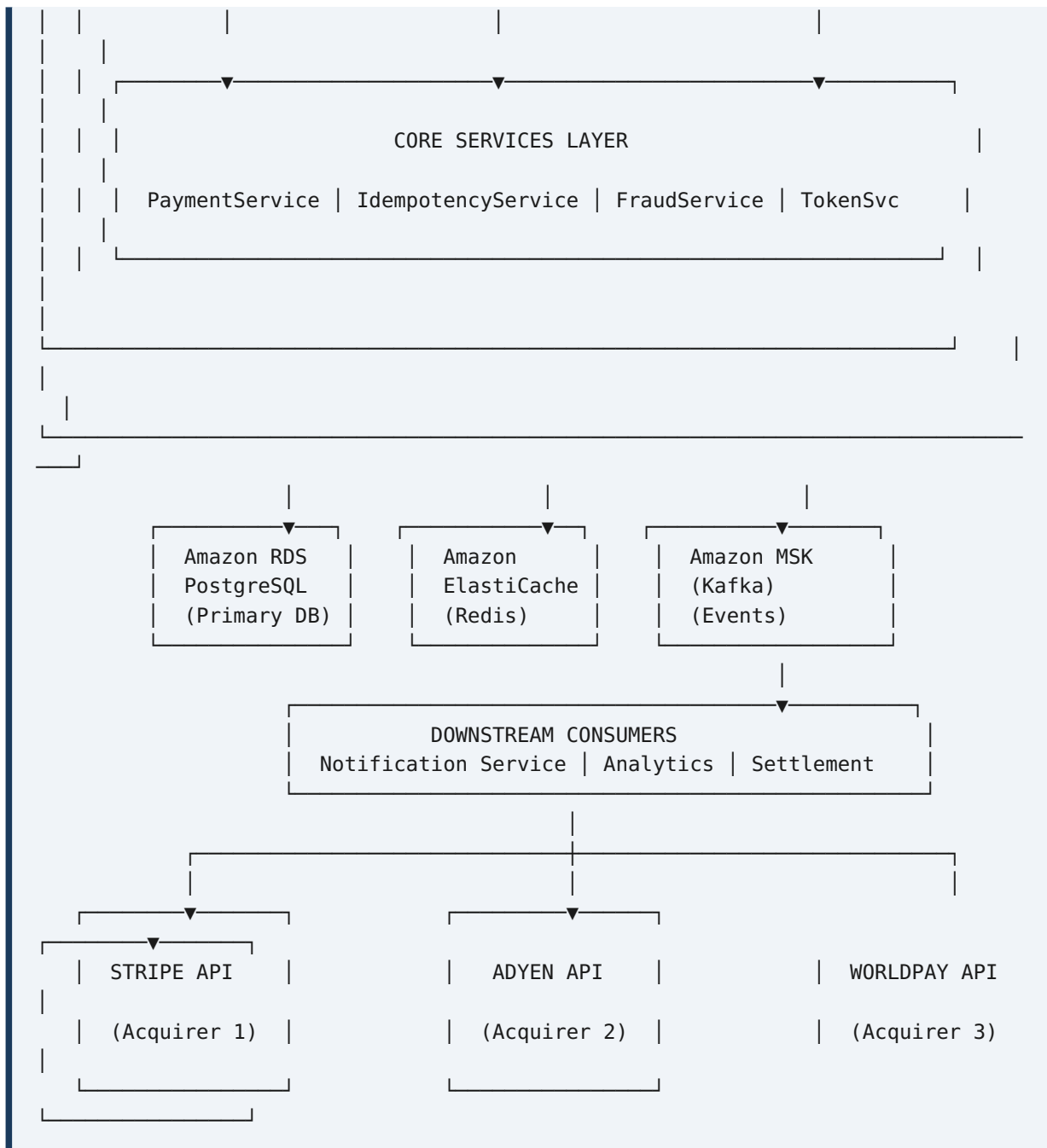
Version: 1.0

Date: 2026-06-08

1. Architecture Overview

The Multi-Acquirer Payment Gateway is built as a **cloud-native microservice** deployed on AWS EKS. It follows a **layered, event-driven architecture** with synchronous APIs for the critical payment path and asynchronous messaging for notifications, settlement, and analytics.





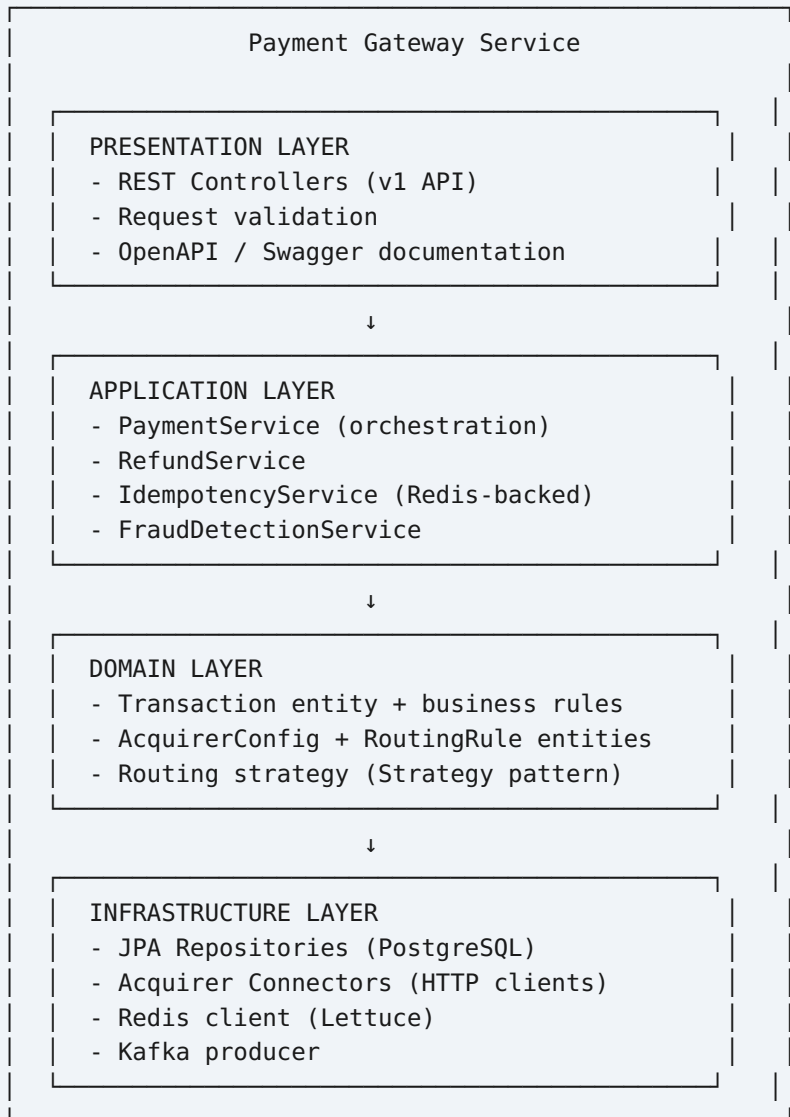
2. Architectural Principles

Principle	Description
API-First	All capabilities exposed via well-documented REST APIs
Idempotency	Every mutating operation supports idempotency keys
Defense in Depth	Security at every layer: network, application, data
Fail-Fast with Fallback	Circuit breakers on all acquirer connections; automatic failover
Observability	Every request traced end-to-end with structured logs and metrics
Zero-Trust	No implicit trust between services; mTLS for internal calls
Immutable Infrastructure	All deployments via container images; no in-place mutations

3. Component Architecture

3.1 Payment Gateway Service (Core)

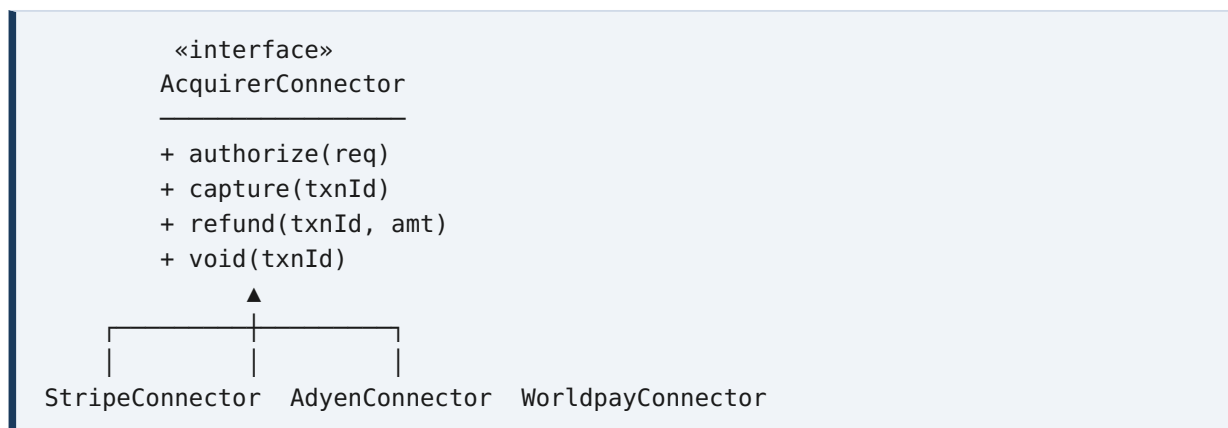
The single Spring Boot microservice containing:



3.2 Routing Engine



3.3 Acquirer Connector Pattern



Each connector wraps: - HTTP client (WebClient with circuit breaker) - Request/response mapping (acquirer-specific ↔ canonical model) - Error classification (hard decline / soft decline / system error)

4. Data Architecture

4.1 Primary Database (PostgreSQL on Amazon RDS)

transactions	acquirer_configs	routing_rules
id (UUID) merchant_id idempotency_key status amount currency card_token acquirer_type acquirer_txn_id created_at updated_at	id acquirer_type api_key_ref (vault) endpoint_url enabled priority success_rate avg_latency_ms	id priority card_network currency min_amount max_amount acquirer_id

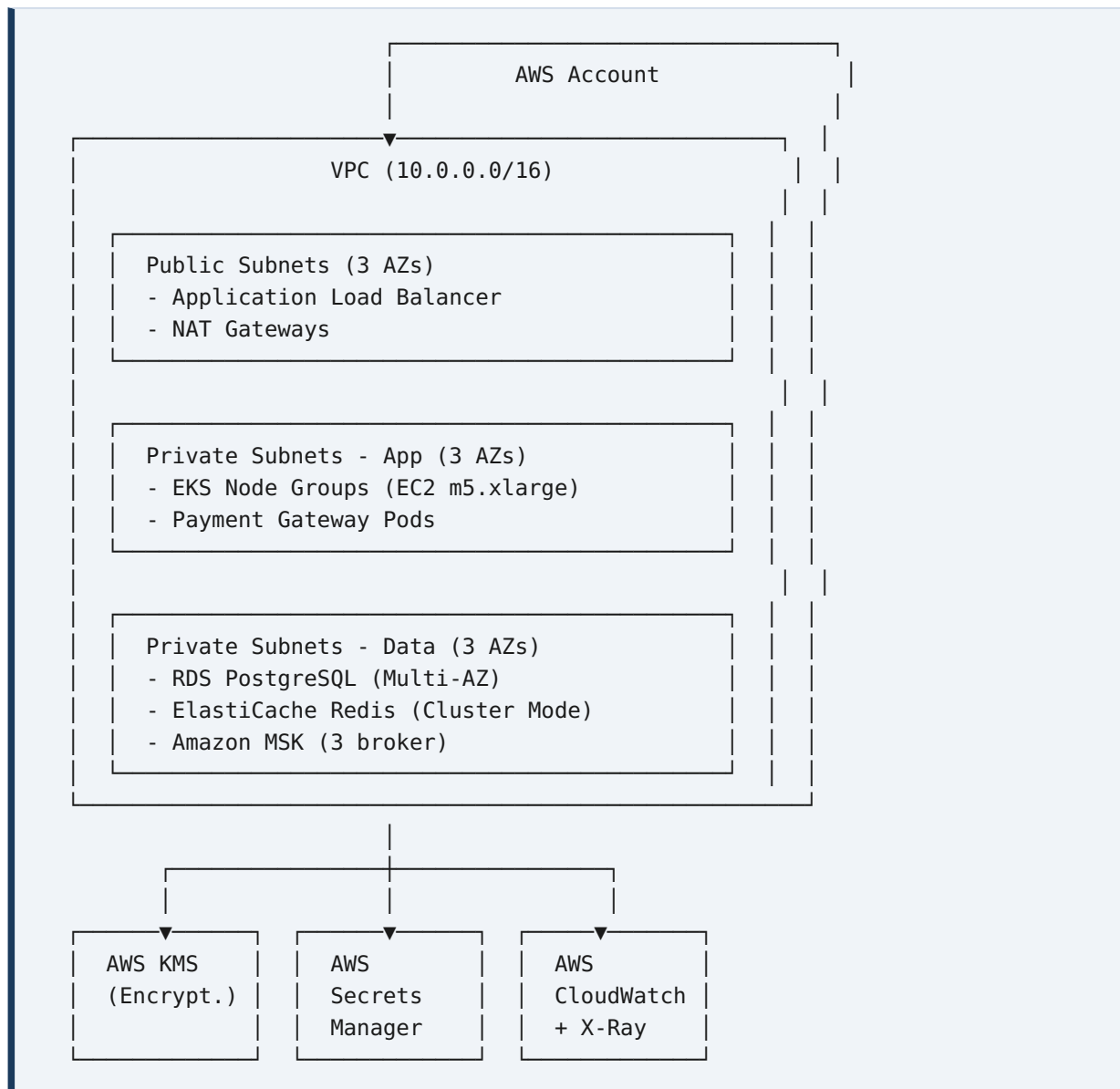
4.2 Cache Layer (Redis on ElastiCache)

Key Pattern	Value	TTL	Purpose
idempotency:{key}	txn_id + status	24h	Duplicate prevention
acquirer:metrics:{id}	JSON metrics	5min	Real-time routing
merchant:{id}:config	routing config	30min	Config caching
session:{token}	merchant session	1h	Auth session

4.3 Event Streaming (Kafka on Amazon MSK)

Topic	Producers	Consumers	Purpose
payment.authorized	Gateway	Notification, Analytics	Authorization events
payment.captured	Gateway	Settlement, Analytics	Capture events
payment.failed	Gateway	Analytics, Alerting	Failure events
payment.refunded	Gateway	Notification, Settlement	Refund events

5. AWS Infrastructure Architecture

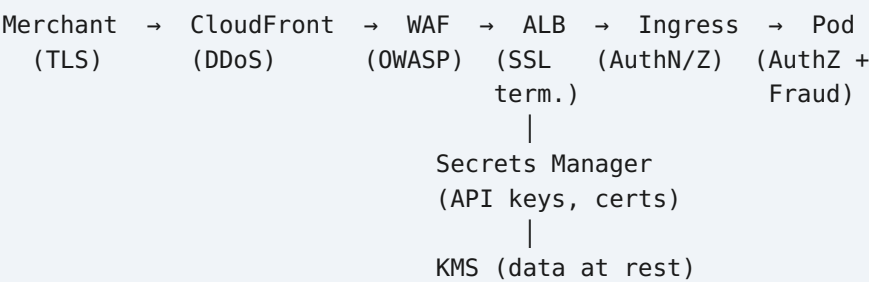


AWS Services Used

Service	Usage	Justification
EKS	Container orchestration	Managed Kubernetes, auto-scaling
RDS PostgreSQL	Transactional data store	ACID compliance, Multi-AZ HA
ElastiCache Redis	Idempotency + caching	Sub-millisecond latency
MSK (Kafka)	Event streaming	Durable, ordered event log
ALB	Load balancing + WAF	SSL termination, path routing
Route 53	DNS + health checks	Failover routing
CloudFront	DDoS + edge caching	Global PoP, Shield Standard
Secrets Manager	API keys, DB passwords	Rotation, audit trail
KMS	Encryption keys	PCI-DSS requirement
CloudWatch	Metrics + logs	Centralized observability
X-Ray	Distributed tracing	Request flow visibility
WAF	Web application firewall	OWASP rule sets

6. Security Architecture

Request Flow with Security Controls:

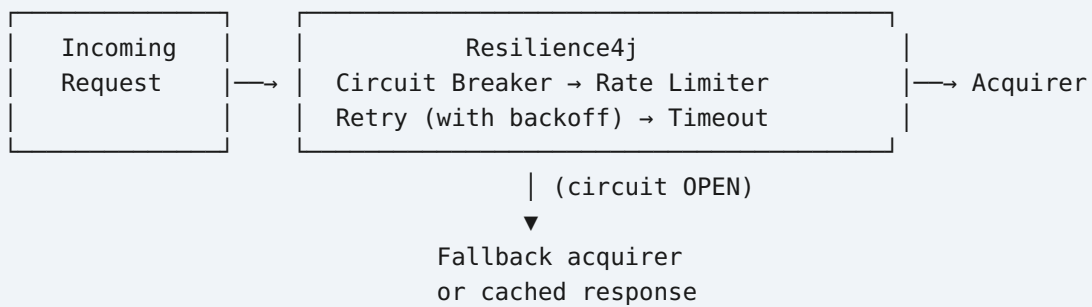


Security Controls by Layer:

Layer	Control
Network	VPC isolation, Security Groups, NACLs, PrivateLink for DB
Edge	AWS WAF (rate limiting, SQLi, XSS rules), Shield Standard
Transport	TLS 1.3 everywhere; mTLS for service-to-service
Application	OAuth2 / API key authentication; per-merchant scopes
Data	AES-256 encryption at rest (KMS); card data tokenization (never stored)
Secrets	AWS Secrets Manager; no credentials in environment vars or code

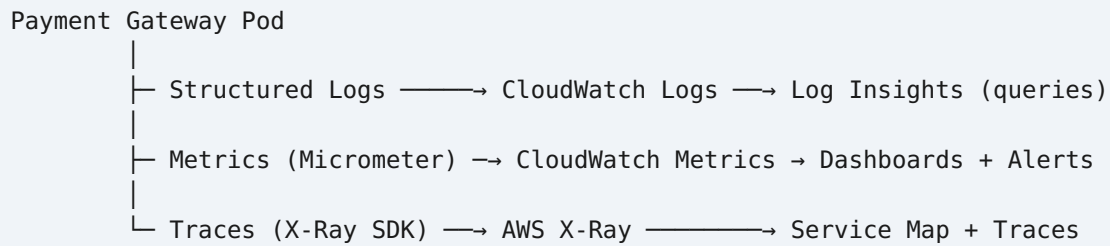
7. Resilience Architecture

Resilience Patterns Applied:



Pattern	Tool	Configuration
Circuit Breaker	Resilience4j	Open after 5 failures in 10s; half-open after 30s
Retry	Resilience4j	2 retries; exponential backoff (100ms, 200ms)
Timeout	Resilience4j	3s per acquirer call
Rate Limiter	Resilience4j	1,000 req/s per merchant
Bulkhead	Resilience4j	Separate thread pools per acquirer
Idempotency	Redis	24h key TTL prevents duplicates

8. Observability Architecture



Key Metrics Tracked: - `payment.authorization.success_rate` (per acquirer, per merchant) - `payment.authorization.latency_p95` (per acquirer) - `payment.routing.acquirer_selected` (routing distribution) - `payment.failover.count` (failover frequency) - JVM metrics: heap, GC, thread pools

9. Deployment Architecture (EKS)

```
EKS Cluster
├─ Namespace: payment-gateway-prod
│  └─ Deployment: payment-gateway (3 replicas min, 10 max)
│  └─ Service: payment-gateway-svc (ClusterIP)
│  └─ Ingress: payment-gateway-ingress (ALB Ingress Controller)
│  └─ HorizontalPodAutoscaler (CPU 70%, Memory 80%)
│  └─ PodDisruptionBudget (minAvailable: 2)
│  └─ ConfigMap: app-config (non-sensitive config)
└─ ServiceAccount (IRSA for Secrets Manager access)
```

End of High-Level Architecture Document